VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Natural Language Processing

CO3086

Assignment Technical Report

Source Code

LEGAL CONTRACT INFORMATION EXTRACTION & SEMANTIC ANALYSIS

Instructor: Mr. Nam Duong Ho

Students: Cuong Doan Phuong Hung ID 2310381
Khiem Ton Vo Hoai ID 2211576
Phong Ha Thuc Thanh ID 2212549

HO CHI MINH CITY, 12 MAY 2026

Contents

I Processing and Syntax Analysis **2**

1 Dataset Overview **2**

2 Clause Splitting **3**
2.1 Objective and Problem Understanding . . . 3
2.2 Preprocessing of Legal Text . . . 4
2.3 Clause Extraction Algorithm . . . 5
2.4 Qualitative Evaluation . . . 7

3 Noun Phrase Chunking **9**
3.1 The Linguistic Challenge in Legal Text . . . 9
3.2 Methodology: The IOB Tagging Schema . . . 10
3.3 Qualitative Evaluation . . . 12

4 Dependency Analysis **13**
4.1 Objective and Syntactic Mapping . . . 13
4.2 Methodology: The Universal Dependencies Framework . . . 13
4.3 Data Representation and JSON Schema . . . 15
4.4 Technical Evaluation . . . 16

II Information Extraction & Semantic Analysis **18**

5 Custom Named Entity Recognition **18**
5.1 Custom Schema Engineering and Justification . . . 18
5.2 Training Dataset Generation & Preprocessing . . . 19
5.2.1 Data Sourcing and Annotation Strategy . . . 19
5.2.2 Sample Annotations . . . 19
5.2.3 Data Validation and Format Conversion . . . 20
5.3 Model Architecture and Fine-Tuning Strategy . . . 21
5.4 Evaluation & Results . . . 24
5.5 Technical Limitations and Trade-offs . . . 26
5.5.1 Domain-Specific Noise and the “Address” Limitation . . . 26
5.5.2 The Trade-off: Precision over Semantic Scope . . . 26
5.5.3 Future Work and Mitigation Strategy . . . 27

6 Semantic Role Labeling **27**

7 Clause Intent Classification **27**

7.1 Intent Schema Definition 28

7.2 Methodology: Hybrid Rule-Based Classification 29

7.3 Technical Evaluation 31

7.4 Challenges and Limitations 32

III Contract Question Answering Chatbot **35**

Abstract

Part I

Processing and Syntax Analysis

1 Dataset Overview

Dataset Selection & Justification

For this project, the primary data source is the **Atticus Open Contract Dataset (CUAD)**. CUAD is a specialized, expert-annotated corpus containing 510 commercial legal contracts. It was created specifically to benchmark Natural Language Processing (NLP) models in the legal domain and provides a highly authoritative collection of unstructured contract text.

While the corpus encompasses over 500 contracts, a full-scale analysis was not necessary for demonstrating the effectiveness of the proposed pipeline. Many of the documents – such as boilerplate Non-Disclosure Agreements – are highly repetitive in structure and lack the complex linguistic content best suited for rigorous evaluation of clause segmentation and extraction algorithms.

Alignment with Assignment Objectives

The selected evaluation documents provide intentionally diverse linguistic challenges that directly correspond to the core milestones of this project. Rather than processing the entire dataset, three contracts were curated to stress-test distinct aspects of the NLP pipeline:

- **Sponsorship Agreement**
  (*iPayment, Inc. and Wells Fargo Bank, N.A.*)
  Syntactic Complexity for Clause Splitting (Section 2): This contract contains multi-tiered definitions and elaborate termination provisions. It is designed to rigorously evaluate the dependency-aware clause segmenter, particularly its handling of subordinating conjunctions and intricate legal enumerations without fragmenting semantic units.

- **Distributor Agreement**
  (*Wireless Links Inc. and Jaguar Investments, Inc.*)
  Technical Density for Noun Phrase Chunking (Section 3): This document is rich in technical multi-word noun phrases such as "G3001 GPSLink mobile terminal" and "MidLink Middleware software." These serve as an exacting test for the IOB tagging procedure required for accurate noun phrase chunking.

- **Software License and Maintenance Agreement**
  (*SFG Financial Corp. and 551 FX IB Associates, LLC*)

Feature-Rich Entities for Custom NER (Section 5): This contract includes a high frequency of intricate numerical and temporal entities – such as precise monetary amounts ($63,309), variable rates ("15% rebate," "tier-based pricing"), and deadlines. These features ensure the NER model is evaluated on a wide array of entity formats.

Additionally, all three selected contracts provide:

- **Explicit Intents for Classification** (Section 7): Clear examples of contractual Obligations ("shall pay"), Prohibitions ("shall not disclose"), and Rights ("reserves the right to"), yielding an unambiguous ground truth for TF-IDF and transformer-based intent classifiers.
- **Structural Depth for RAG Implementation** (Assignment III): Extensive appendices, schedules, and cross-referenced sections offer a realistic test bed for Retrieval-Augmented Generation, requiring high-precision retrieval of specific fee schedules and penalty stipulations.

This deliberately engineered subset, spanning a spectrum of commercial contract types and linguistic features, ensures comprehensive evaluation of clause segmentation, NER, classification, and retrieval capabilities.

2 Clause Splitting

2.1 Objective and Problem Understanding

Legal contracts typically consist of long, complex sentences that include multiple clauses connected through conjunctions, conditions, and enumerations. These structures make downstream Natural Language Processing (NLP) tasks – such as entity recognition or semantic role labeling – significantly more difficult. Therefore, the goal of this task is to transform raw contract text into a sequence of semantically independent clauses, where each clause represents a meaningful and analyzable unit.

To achieve this, the implementation focuses on designing a hybrid rule-based and dependency-aware clause segmentation algorithm that leverages both surface-level punctuation and syntactic structure.

Approach Overview:

- **Rule-Based Splitting:** Initial clause candidates are extracted using heuristics based on surface features such as semicolons, periods, and conjunctions. These are strong signals of clause and sentence boundaries in legal English.

- **Dependency Parsing:** For challenging cases where punctuation is absent or ambiguous, a syntactic dependency parser identifies clausal structures in complex sentences.
- **Hybrid Postprocessing:** The outputs from both strategies are further refined. For example, enumerations and subordinate clauses attached to the main clause are separated to ensure semantic independence.

This combination of shallow and deep linguistic cues helps ensure that each segment produced is both syntactically and semantically meaningful for downstream tasks.

2.2 Preprocessing of Legal Text

Before clause splitting, the raw contract text is subjected to a dedicated preprocessing stage designed to clean and normalize the data. Legal documents, particularly those sourced from public filings, frequently contain artifacts such as redaction markers, extraneous page headers/footers, and inconsistent whitespace. If left unaddressed, these features can negatively impact the performance of subsequent NLP analyses – especially clause segmentation.

```python
def clean_legal_text(text):
  """Removes redactions, page numbers, and formatting artifacts."""
  text = re.sub(r'\[\*\*\*\]', '', text)
  text = re.sub(r'Page \d+', '', text)
  text = re.sub(r'\n+', ' ', text)
  text = re.sub(r'\s+', ' ', text)
  return text.strip()
```

The preprocessing function applies several key transformations:

- **Removal of redaction markers:** Textual placeholders such as `[***]`, which indicate redacted or confidential content, are stripped from the document to prevent confusion with actual clause content.
- **Elimination of page numbering:** Regular expressions are used to identify and remove page headers or footers (e.g., "Page 1") that appear throughout scanned or OCR-processed contracts.
- **Whitespace normalization:** Multiple consecutive newline (`\\n`) and space characters are replaced with a single space, ensuring that sentence and clause boundaries are not obscured by formatting noise.

Through these steps, the input text is transformed into a linguistically consistent form that is more amenable to parsing, tokenization, and clause boundary detection. The preprocessing routine helps maximize the precision and recall of the clause splitter by minimizing spurious tokens and misplaced boundaries in the downstream analysis.

2.3 Clause Extraction Algorithm

At the core of the system is the `extract_clauses()` function, responsible for identifying clause boundaries within the preprocessed legal text. The method utilizes spaCy's NLP pipeline to perform tokenization and dependency parsing, iterating sequentially over each token to dynamically determine appropriate split points based on linguistic and syntactic cues. To handle the unique complexities of legal texts, the algorithm is divided into three specialized heuristic rules.

Algorithm 1 Dependency-Aware Clause Splitting

```
procedure ExtractClauses(raw_text)
    doc ← PreprocessAndParse(raw_text)
    clauses ← [ ], start_pos ← 0
    for each token in doc do
        split ← false
        if token ∈ {".",";"} then                                  ▷ Rule 1: Hard Punctuation
            split ← true
        else if token = ":" and Next(token) = "(" then             ▷ Rule 2: Look-Ahead
            split ← true
        else if token.dep = "cc" and IntroducesNewSubject(token) then   ▷ Rule 3: Syntactic
            split ← true
        end if
        if split then
            span ← Text from start_pos to token (inclusive)
            PostProcessAndStore(clauses, span)
            start_pos ← position after token
        end if
    end for
    return clauses
end procedure
```

Token Whitespace Preservation

A critical challenge in NLP text segmentation is the **Tokenization Trap**. When parsers like spaCy process text, they split punctuation marks, symbols, and words into discrete tokens. Naively reconstructing a clause by joining these tokens with spaces ruins the original formatting, yielding artifacts such as `( i )` instead of `(i)` or "V / MC Systems" instead of "V/MC Systems".

In the legal domain, preserving exact formatting is paramount. To resolve this, the algorithm abandons token-level string joining. Instead, it maintains index pointers (`start_idx` and `end_idx`) and slices the original document span (`doc[start_idx:end_idx].text`). Because spaCy spans retain the exact character offsets of the raw input, this slicing technique guarantees 100% fidelity to the original contract formatting, eliminating artificial whitespace while ensuring

clean data for downstream tasks.

Handling Legal Enumerations

Standard sentence boundary detection models are trained on general English, where periods indicate the end of a thought. However, legal contracts frequently utilize complex enumerations, where massive lists of conditions or obligations are chained together using semicolons (;). A standard tokenizer will fail on these structures, grouping multiple distinct conditions into a single, unmanageable paragraph.

To address this, the algorithm explicitly defines the semicolon as a hard clause boundary. This isolates each condition into its own independent string.

Standard Parser Output (Failure):

> *"Such Reserve Account may be funded by: (i) one or more debits; (ii) one or more deductions; or (iii) ISO's delivery of a letter of credit."*
> (Treated as one massive clause).

Our Algorithm Output (Success):

Clause 1. Such Reserve Account may be funded by: (i) one or more debits;

Clause 2. (ii) one or more deductions;

Clause 3. or (iii) ISO's delivery of a letter of credit.

By splitting explicitly at semicolons, the algorithm preserves the semantic independence of each list item. This is critical for Intent Classification task (Section 7), as it allows the classifier to evaluate each condition individually rather than attempting to classify a convoluted block of text.

Syntactic Dependency Rules

For complex sentences where punctuation is insufficient or ambiguous (such as clauses connected by "and", "or", or "but"), the algorithm leverages dependency parsing to determine if a split is syntactically valid. Simply splitting at every coordinating conjunction (`cc`) results in severe over-splitting, destroying compound adjectives or compound predicates (e.g., splitting "Approval will not be withheld or delayed" into two meaningless fragments).

To prevent this, a strict **"New Subject" Heuristic** was implemented. The algorithm will only split a sentence at a conjunction if *all* of the following syntactic conditions are met:

- The token is a coordinating conjunction (`dep_ == "cc"`).

- Its head is a verb (`pos_ == "VERB"`).
- The head verb is connected to a secondary verb (`dep_ == "conj"`).
- **Critical Check:** The secondary verb possesses its own explicit nominal subject (`nsubj` or `nsubjpass`).

Example of the heuristic in action:

> "*In the event a Merchant refuses to consent, ISO will notify SERVICERS, and Bank will have the right to terminate its sponsorship.*"

The parser detects the conjunction "and", identifies "will notify" and "will have" as the two connected verbs, and verifies that "Bank" is the explicit subject of the second verb. Because a new subject is present, it is a truly independent clause, and the algorithm safely cleanly severs the string before the conjunction.

2.4 Qualitative Evaluation

The effectiveness of the Dependency-Aware Clause Splitter is evaluated by comparing the raw OCR-processed text of the Sponsorship Agreement against the final segmented output. This qualitative assessment focuses on three primary linguistic successes: governing clause isolation, enumeration precision, and syntactic integrity.

```
10.3  Termination by ISO. ISO may terminate this Agreement prior to its expiration for cause upon prior written notice
to SERVICERS as follows:

  (a)   Upon a Material breach of any representation, warranty or covenant in this Agreement by SERVICERS which is not
  cured by SERVICERS within sixty (60) days of receipt of written notice from ISO; provided however, that if SERVICES
  are attempting in good faith to cure such breach within said sixty (60) day cure period but the nature of the breach
  prevents a cure within sixty (60) days, then SERVICERS shall be allotted an additional number of days to cure,
  provided the total number of days of the cure period shall be no greater than one hundred twenty (120) days;

  (b)   Upon any failure by SERVICERS to pay any amount due under this Agreement which does not give rise to the right
  to terminate under any other provision of this Agreement within five (5) Business Days after written notice to
  SERVICERS of its failure to pay the amount;

  (c)   Upon the insolvency or bankruptcy of either of the SERVICERS;

  (d)   Upon the termination of the Service Agreement for any reason;

  (e)   Upon a Material adverse change in the business or financial condition of either of SERVICERS;

  (f)   Upon an assignment of this Agreement by SERVICERS without ISO's prior written consent;

  (g)   Upon the failure by SERVICERS to maintain good standing with Visa or a MasterCard;

  (h)   If required by a governmental or regulatory body or agency or by any Bank Card association to terminate;

  (i)   Upon cessation of Bank Card operations by SERVICERS; or
```

Figure 2.1: Original OCR-based Sponsorship Agreement

Figure 2.1 presents a raw, unsegmented snapshot of Section 10.3 from the Sponsorship Agreement as it appears post-OCR processing. This image serves as a *high-difficulty baseline*, reflecting the dense, non-linear structure typical of commercial legal filings.

The section is anchored by a "governing clause" that transitions directly into a complex enumeration of ten distinct termination triggers, each separated by semicolons and frequently containing nested conditions – notably the "provided however" block in item (a). This input exemplifies the so-called "Tokenization Trap": conventional sentence splitters typically fail, either merging all ten items into a single run-on clause or fragmenting the logic of subordinate phrases (such as conditions within enumerated items).

By adopting this particular passage as the starting point, the evaluation below underscores the limitations of off-the-shelf sentence segmentation and illustrates the necessity of the proposed hybrid, dependency-aware heuristic to preserve both syntactic boundaries and conditional logic.

```
377   10.3 Termination by ISO.
378   ISO may terminate this Agreement prior to its expiration for cause upon prior written notice to SERVICERS as follows:
379   (a) Upon a Material breach of any representation, warranty or covenant in this Agreement by SERVICERS which is not cured by SERVICERS within
      sixty (60) days of receipt of written notice from ISO;
380   provided however, that if SERVICES are attempting in good faith to cure such breach within said sixty (60) day cure period but the nature of
      the breach prevents a cure within sixty (60) days, then SERVICERS shall be allotted an additional number of days to cure, provided the total
      number of days of the cure period shall be no greater than one hundred twenty (120) days;
381   (b) Upon any failure by SERVICERS to pay any amount due under this Agreement which does not give rise to the right to terminate under any
      other provision of this Agreement within five (5) Business Days after written notice to SERVICERS of its failure to pay the amount;
382   (c) Upon the insolvency or bankruptcy of either of the SERVICERS;
383   (d) Upon the termination of the Service Agreement for any reason;
384   (e) Upon a Material adverse change in the business or financial condition of either of SERVICERS;
385   (f) Upon an assignment of this Agreement by SERVICERS without ISO's prior written consent;
386   (g) Upon the failure by SERVICERS to maintain good standing with Visa or a MasterCard;
387   (h) If required by a governmental or regulatory body or agency or by any Bank Card association to terminate;
388   (i) Upon cessation of Bank Card operations by SERVICERS;
389   or (j) At any time upon one hundred eighty (180) days prior written notice to the SERVICERS with no penalty.
```

Figure 2.2: Clause Splitting Output

2.5.1 Success in Structural Segmentation

As evidenced by the IDE line numbers in Figure 2.2, each clause is successfully isolated onto an independent line, proving that the segmentation logic correctly terminates strings at identified boundaries.

- **Governing Clause Isolation:** The algorithm correctly applied Rule 2 (Look-Ahead) at Section 10.3. It identified the colon after "as follows:" and severed the string before the start of the enumeration (a).
- **Semantic Independence:** By isolating the governing intent ("ISO may terminate...") from the specific triggers, the pipeline ensures the Intent Classifier (Section 7) can distinguish between a "Right" and its "Conditions".

2.5.2 Handling Deep Enumerations

The evaluation confirms a high degree of precision in handling Rule 1 (Hard Punctuation) boundaries within legal lists.

- **Triggers (a) through (j):** Each termination condition is extracted as a distinct semantic unit.
- **Data Balancing:** Splitting at the semicolon prevents the creation of a "heavy first item," which occurs when introductory text is erroneously merged with the first list entry. This leads to more balanced feature vectors for downstream machine learning tasks.

2.5.3 Syntactic Robustness

The most rigorous test of the New Subject Heuristic (Rule 3) appears in clause (a), which contains a complex "provided however" condition.

- **Constraint Success:** Despite the presence of multiple potential split points – including "and", "but", and "then" – the algorithm correctly maintained the entire 60-day to 120-day cure period as a single, contiguous clause.
- **Logic:** Because the algorithm requires a new nominal subject to trigger a split, it recognized these fragments as dependent modifiers of the primary breach rather than independent actions.

Through these results, the system demonstrates that it can navigate the dense formatting of the Atticus Open Contract Dataset while preserving 100% fidelity to the original text via span slicing.

3 Noun Phrase Chunking

Following the isolation of semantically independent clauses in Section 2, the next critical phase in the NLP extraction pipeline is **Noun Phrase (NP) Chunking**. The primary objective of this task is to identify and extract non-recursive noun phrases from the segmented text, representing their boundaries using the standard **Inside-Outside-Beginning (IOB)** tagging schema.

3.1 The Linguistic Challenge in Legal Text

In general domain text, entities are often single words or simple bigrams (e.g., "the company"). However, **commercial contracts exhibit extreme technical density**, making entity extraction significantly more complex. As highlighted in the dataset overview, documents such

as the Distributor Agreement frequently contain entities realized as complex, multi-word syntactic constituents – for example, "G3001 GPSLink mobile terminal" or "MidLink Middleware software."

If an NLP parser fails to recognize these multi-word strings as a single noun phrase, it will fracture the entity. For example, a naive parser might tag "mobile" as an adjective and "terminal" as a noun, losing the fact that the entire four-word string represents a single, indivisible hardware product governed by the contract.

Strategic Importance to the Pipeline

Noun Phrase Chunking is not merely a formatting exercise; it is a foundational prerequisite for the advanced machine learning objectives in Assignment II:

- **Semantic Boundary Definition** (Section 5): By grouping determiners, adjectives, and nouns into a single chunk, the algorithm establishes the exact token boundaries required to train a *Custom Named Entity Recognition* model on intricate entities (e.g., complex product names, precise numerical and temporal phrases).
- **Contextualizing Obligations** (Section 7): While Clause Splitting (Task 1.1) isolates what the contractual rule is (the predicate), Noun Phrase Chunking isolates the *actors* and *objects* involved. It identifies who is obligated (e.g., "SERVICERS") and what specific object or payment is being acted upon (e.g., "Net Program Participation Fees").

By accurately mapping these phrases into the `Token \t Tag` tabular format required by the problem specification, the pipeline effectively transforms unstructured linguistic text into discrete, machine-readable objects, ready for entity classification and downstream retrieval.

3.2 Methodology: The IOB Tagging Schema

The objective of this module is to identify key entities – such as contracting parties, financial amounts, and legal deadlines – by labeling noun phrases (NPs) within extracted clauses. To structure the extracted clauses for downstream entity recognition, the pipeline implements the **Inside-Outside-Beginning (IOB)** tagging schema. This standard sequence-labeling format allows the system to represent the precise boundaries of multi-word noun phrases (NPs) at the token level.

Algorithm 2 IOB Noun Phrase Chunking with Boundary Reset

```
procedure GenerateIOBTags(clause)
    doc ← NLP_Parse(clause)
    tags ← array of "O" with length equal to number of tokens in doc
    for each chunk in doc.noun_chunks do
        start_new ← true
        for each token in chunk do
            if IsExcluded(token) then                    ▷ Filter punct, preps, etc.
                tags[token] ← "O"
                start_new ← true                         ▷ Reset NP boundary
            else
                tags[token] ← start_new ? "B-NP" : "I-NP"
                start_new ← false
            end if
        end for
    end for
    return FormatOutput(doc, tags)
end procedure
```

Dependency-Based Chunking Instead of using a simple Regular Expression (RegEx) parser, we implemented a methodology leveraging spaCy's dependency parser. This allows the system to:

- **Handle Complex Legal Syntax:** Legal text often contains nested modifiers. By utilizing the `noun_chunks` iterator, we identify phrases based on their syntactic "head" noun rather than just linear word order.
- **IOB Tagging Scheme Implementation:** We map the detected chunks to the required IOB format:
  - **B-NP (Beginning of Noun Phrase):** This tag is exclusively assigned to the very first token of a recognized noun phrase. It signals to the downstream model that a new entity or object boundary has started.
  - **I-NP (Inside a Noun Phrase):** This tag is assigned to all subsequent tokens that belong to the same noun phrase initiated by the B-NP tag.
  - **O (Outside):** This tag is assigned to any token that does not belong to a noun phrase. In legal contracts, this typically includes verbs, prepositions, coordinating conjunctions, and hard punctuation.

Refinement and Edge Case Handling To meet the specific requirements of legal document processing, the following logic adjustments were made:

- **Punctuation Filtering:** Standard legal punctuation (e.g., commas, periods, hyphens) is forced to the O tag to maintain clean boundaries between distinct legal entities.

- **Strict Boundary Resets:** We implemented a "reset" mechanism where any token following an O tag must start with B-NP if it belongs to a new noun phrase. This prevents "broken" sequences where an I-NP incorrectly follows an O tag.
- **Financial and Mathematical Symbols:** Special handling was applied to currency symbols (e.g., $) and operators (e.g., =) to ensure financial calculations and amounts remain semantically intact within their respective chunks.

3.3 Qualitative Evaluation

The effectiveness of the dependency-aware Noun Phrase Chunking module is evaluated by examining its ability to maintain syntactic continuity across complex legal ownership structures. Traditional shallow parsers frequently fracture multi-word entities when encountering punctuation, coordinating conjunctions, or possessive markers. To demonstrate the robustness of the implemented IOB tagging schema, the following sample from the *Sponsorship Agreement* illustrates the algorithm's handling of chained possessives and coordinated actors:

Token	Merchant	's	or	ISO	's	participation	in	MasterCard
IOB Tag	B-NP	I-NP	I-NP	I-NP	I-NP	I-NP	O	B-NP

Syntactic Continuity and Possessive Chaining As evidenced by the output above, the parser successfully maps the entire phrase `Merchant's or ISO's participation` as a single, contiguous semantic unit.

- **Possessive Integrity:** The algorithm correctly identifies that the possessive suffixes (`'s`) are not independent tokens to be excluded, but rather integral modifiers linking the contracting parties (`Merchant`, `ISO`) to the target object (`participation`). Tagging them as I-NP ensures the "Owner" and "Owned Object" remain syntactically bound.
- **Coordination Retention:** By tagging the conjunction `or` as I-NP, the pipeline avoids splitting the entity into two fragmented objects. This prevents `participation` from floating without its full ownership context, which is a vital prerequisite for accurately training the Named Entity Recognition model in Section 5.

Solving the "Broken Chunk" Problem Furthermore, the implementation guarantees strict sequence adherence through its boundary reset logic. In standard rule-based chunking, filtering out specific tokens (such as prepositions or hard punctuation) often leads to the "broken chunk" problem, where an I-NP tag incorrectly immediately follows an O tag, invalidating the entity boundaries for downstream machine learning models.

By dynamically resetting the beginning tag after any excluded token, the algorithm ensures that every newly initiated noun phrase strictly begins with a B-NP. The transition from

`participation` (I-NP) to `in` (O) and immediately to `MasterCard` (B-NP) proves that the extraction sequence is fully sanitized, resolving the broken chunk vulnerability and generating an artifact that is perfectly aligned with standard sequence-labeling requirements.

4 Dependency Analysis

4.1 Objective and Syntactic Mapping

The final component of the Assignment 1 NLP pipeline is **Dependency Analysis**. While Clause Splitting (Section 2) isolated the semantic boundaries of the text and Noun Phrase Chunking (Section 3) identified the specific entities involved, the objective of Dependency Analysis is to uncover the grammatical skeleton connecting them. This task extracts and maps the word-to-word *head-dependent* relationships for every token within a given clause, creating a structured tree of syntactic dependencies.

The Linguistic Utility in Legal English

Commercial contracts are notoriously dense, frequently employing passive voice, deep embedding, and vast distances between a governing verb and its associated subjects or objects. For instance, in a complex termination clause, a shallow keyword search or simple Part-of-Speech (POS) tagger might struggle to map the exact relationship between the acting party and the action due to the sheer volume of surrounding legal modifiers and conditions.

Dependency mapping resolves this ambiguity by translating the linear text into a directed graph of the sentence syntax. By identifying the root predicate (`ROOT` = "terminate") and tracing its directed edges to the nominal subject (`nsubj` = "ISO") and the direct object (`dobj` = "Agreement"), the pipeline exposes the core action of the clause regardless of word distance, intervening noise, or convoluted phrasing.

This token-level syntactic mapping is not only a formal requirement of the project but also a critical necessity for advanced relation extraction. It provides the exact structural logic required to populate the target JSON schema, enabling downstream machine learning models to programmatically understand who is doing what to whom within the contract.

4.2 Methodology: The Universal Dependencies Framework

To programmatically extract the grammatical relationships within the segmented clauses, the pipeline utilizes the **Universal Dependencies (UD)** framework. UD is a standardized, cross-linguistic annotation schema that categorizes syntactic relationships between a "head" token and its syntactic "dependents."

Algorithm 3 Syntactic Dependency Extraction and JSON Serialization

```
procedure GenerateDependencyJSON(clauses)
    dataset_array ← [ ]
    for each clause in clauses do
        dependencies ← ExtractDependencies(clause)
        clause_record ← {"Clause": clause, "Dependencies": dependencies}
        Append(dataset_array, clause_record)
    end for
    return dataset_array
end procedure
procedure ExtractDependencies(clause)
    doc ← DependencyParse(clause)                    ▷ Generate directed syntactic graph
    token_mappings ← [ ]
    for each token in doc do
        if token.text.strip() ≠ "" then              ▷ Skip whitespace and empty tokens
            dep_entry ← {
                "Token": token.text,
                "Head": token.head.text,             ▷ String form of syntactic governor
                "Dependency": token.dep_label
            }
            Append(token_mappings, dep_entry)
        end if
    end for
    return token_mappings
end procedure
```

The Parsing Engine

The implementation leverages spaCy's `en_core_web_sm` model, which features a highly optimized, transition-based dependency parser. Unlike traditional constituency parsing – which breaks sentences into nested phrase brackets – transition-based dependency parsing establishes direct, directed arcs between individual words. During execution, the parser analyzes the topological structure of the clause and assigns a specific syntactic dependency label to every token. Key labels crucial for legal text analysis include:

- **ROOT**: The primary predicate or main verb of the clause (e.g., "terminate", "warrant", "pay"). All other words in the sentence ultimately trace their dependency back to this root.
- **nsubj / nsubjpass**: The nominal subject (or passive subject) performing the action, which usually identifies the obligated or acting party (e.g., "ISO", "SERVICERS").
- **dobj / pobj**: The direct object of a verb or the object of a preposition, typically identifying the target of an action or rule (e.g., "Agreement", "fees").
- **amod**: Adjectival modifiers that define the exact scope of legal entities (e.g., "prior" and "written" modifying "notice").

Handling Legal Modifiers and Complexity

Commercial contracts are characterized by extreme syntactic depth, heavily relying on chained prepositional phrases and clausal complements (e.g., "in accordance with the terms of this Agreement").

The dependency-based methodology excels in this environment because it systematically decomposes these chains without losing the governing logic. The parser links prepositions (`prep`) to their governing verbs or nouns, and then maps them to their subsequent prepositional objects (`pobj`). By mapping these relationships as a directed graph, the pipeline ensures that subordinate conditions and complex legal modifiers do not break the analytical chain. This allows the system to trace a complete syntactic pathway from a deep modifier all the way back to the governing `ROOT` verb, capturing the full structural reality of the clause.

4.3 Data Representation and JSON Schema

To transform the directed acyclic graph (DAG) generated by the dependency parser into a serializable format, the system implements a mapping function that iterates through the parsed text. Each token is converted into a structured entry within a JSON array, ensuring that the syntactic hierarchy is preserved through explicitly defined `Head` pointers.

The Mapping Logic

- **Serialization:** The implementation captures the literal string of each token (`Token`) and its immediate syntactic governor (`Head`). By representing the head as its string form (not as a numeric index), the structure remains human-readable and easily verifiable.
- **Labeling:** The `Dependency` field utilizes standardized Universal Dependencies labels (such as `nsubj`, `ROOT`), explicitly codifying the grammatical relation for each token.
- **Noise Reduction:** Before converting to JSON, the extraction process skips over any non-semantic tokens (such as whitespace or stray newlines) using a simple filter (`if token.text.strip():`). This prevents OCR or formatting artifacts from polluting the output.

Output Structure

The final output aggregates this information into a nested JSON array, where each entry contains a segmented clause and a list of its token-level dependency mappings:

```
{
  "Clause": "ISO may terminate this Agreement",
  "Dependencies": [
    { "Token": "ISO",       "Head": "terminate", "Dependency": "nsubj" },
    { "Token": "may",       "Head": "terminate", "Dependency": "aux"   },
    { "Token": "terminate", "Head": "terminate", "Dependency": "ROOT"  },
    { "Token": "this",      "Head": "Agreement", "Dependency": "det"   },
    { "Token": "Agreement", "Head": "terminate", "Dependency": "dobj"  }
  ]
}
```

This dictionary-based representation fulfills the data formatting requirements and provides a clean, machine-readable structure that supports both human audit and downstream automation for retrieval or relation extraction.

4.4 Technical Evaluation

The evaluation of the Dependency Analysis module focuses on two primary metrics: **syntactic fidelity** in complex legal sentences and **strict compliance** with the required JSON data schema.

Syntactic Fidelity and Structural Accuracy

The dependency serialization is illustrated below with this sample output, parsed from "The confidential portion has been filed separately with the SEC.":

```
{
  "Clause": "The confidential portion has been filed separately with the SEC.",
  "Dependencies": [
      { "Token": "The",          "Head": "portion", "Dependency": "det"        },
      { "Token": "confidential", "Head": "portion", "Dependency": "amod"       },
      { "Token": "portion",      "Head": "filed",   "Dependency": "nsubjpass"  },
      { "Token": "has",          "Head": "filed",   "Dependency": "aux"        },
      { "Token": "been",         "Head": "filed",   "Dependency": "auxpass"    },
      { "Token": "filed",        "Head": "filed",   "Dependency": "ROOT"       },
      { "Token": "separately",   "Head": "filed",   "Dependency": "advmod"     },
      { "Token": "with",         "Head": "filed",   "Dependency": "prep"       },
      { "Token": "the",          "Head": "SEC",     "Dependency": "det"        },
      { "Token": "SEC",          "Head": "with",    "Dependency": "pobj"       },
      { "Token": ".",            "Head": "filed",   "Dependency": "punct"      }
  ]
}
```

This output demonstrates how the parser preserves the grammatical architecture of a legal clause:

– `det`: "The" is a determiner for "portion".

- `amod`: "confidential" is an adjectival modifier of "portion".
- `nsubjpass`: "portion" is identified as the passive subject of "filed".
- `aux/auxpass`: "has" and "been" are auxiliary elements of the passive construction.
- `ROOT`: "filed" is marked as the root verb of the sentence.
- `advmod`: "separately" acts as an adverbial modifier of "filed".
- `prep`: "with" introduces the prepositional phrase attached to "filed".
- `det`: "the" determines "SEC" within the prepositional object.
- `pobj`: "SEC" is the object of the preposition "with".
- `punct`: "." is recognized as punctuation attached to the root.

By providing each clause and its full dependency breakdown in this machine-readable JSON structure, the parser enables high-precision syntactic and semantic analysis suitable for contract understanding and automation.

Schema Compliance and Noise Robustness

The serialization module proved highly robust when converting the topological graph into the required flat JSON format. Every valid token is successfully mapped to the `Token`, `Head`, and `Dependency` keys without any data loss. Crucially, the implementation of the whitespace filtering heuristic effectively neutralized OCR layout artifacts. By enforcing a strict strip check before appending to the dictionary, stray newline characters, tab indents, and non-breaking spaces from the raw CUAD data were completely excluded from the final syntactic dataset.

The resulting JSON arrays provide a pristine, machine-readable foundation. This output strictly fulfills the final structural requirement discussed in Section 2, cleanly concluding the Processing and Syntax Analysis phase and preparing the text pipeline for the Machine Learning and Classification objectives in Assignment II.

Part II

Information Extraction & Semantic Analysis

5 Custom Named Entity Recognition

General-purpose Named Entity Recognition (NER) systems are highly effective for standard natural language but consistently underperform when applied to the dense, idiosyncratic domain of legal contracts. Standard NER models frequently fragment long corporate names, fail to distinguish between generic organizations and legally bound parties, and lack the domain-specific vocabulary to identify critical structural references (e.g., internal sections, exhibits, or governing policies). In complex documents, this lack of syntactic and semantic precision prevents the accurate extraction of "who does what," rendering the data unusable for downstream automated reasoning or Question Answering (QA) tasks.

Objective

The primary objective of this task is to develop a domain-specific NER pipeline by fine-tuning a pre-trained language model (`en\_core\_web\_md` via spaCy). This pipeline transforms unstructured legal clauses into structured, machine-readable representations by accurately identifying and extracting contract-specific entities.

5.1 Custom Schema Engineering and Justification

The project specification notes that the entity schema "may include (but is not limited to)" financial and penal variables such as `MONEY`, `RATE`, and `PENALTY`. However, an exploratory analysis of the specific corpus selected for this task – a complex Sponsorship Agreement involving multiple corporate entities – revealed that financial variables were too sparse to generate a robust training signal. Attempting to force the model to learn these sparse variables resulted in severe hallucinations and misclassification of standard text.

Therefore, to maximize extraction precision and align with the actual semantic reality of the selected contract, a highly tailored custom schema was engineered. The document's core logic relies heavily on policy adherence, structural cross-referencing, and party obligations rather than transactional financial penalties. The final schema is defined as follows:

PARTY: Any legally defined company, organization, or role. These are strictly modeled as continuous spans to prevent entity fragmentation.

AGREEMENT: Explicit references to governing documents.

SECTION: Internal structural references necessary for tracing contractual logic.

EXHIBIT: External structural references, attachments, and schedules.

DATE: Timeframes, deadlines, and specific temporal milestones, including relative legal dates.

POLICY: Governing rules, standards, and guidelines that dictate party behavior and compliance.

By aligning the NER schema directly with the architectural reality of the specific contract, this pipeline prioritizes high-precision extraction of structural and relational entities, establishing a reliable data foundation for subsequent Semantic Role Labeling and intent classification.

5.2 Training Dataset Generation & Preprocessing

5.2.1 Data Sourcing and Annotation Strategy

The training dataset was constructed by manually extracting and annotating complex clauses from the core Sponsorship Agreement. To ensure the model learned precise entity boundaries without being confused by domain-specific noise, strict annotation guidelines were enforced:

- **Atomic Entity Spans:** Corporate entities and legal names were annotated as single, continuous spans – for example, separating "Wells Fargo Bank, N.A." from "First Data Merchant Services Corporation" rather than grouping them.
- **Exclusion of Variable Noise:** Highly irregular contact information (such as physical addresses, zip codes, and facsimile numbers) were deliberately tagged as O (*Outside*) to prevent the model from misclassifying them as `SECTION` or `PARTY` entities.

5.2.2 Sample Annotations

The dataset was formatted into JSON dictionaries, mapping the raw text to character-level entity spans `[start, end, LABEL]`. Below are representative samples from the training data that illustrate the application of the custom schema:

Sample 1: Temporal and Structural Logic This sample demonstrates the model's training for identifying structural references and relative legal timeframes.

```
{
  "text": "Merchant agrees to comply with the Rules and the Merchant Processing Policy
  ↪  within ninety (90) days of the Effective Date.",
```

```
  "entities": [
    [0, 8, "PARTY"],     // Merchant
    [35, 40, "POLICY"], // Rules
    [49, 75, "POLICY"], // Merchant Processing Policy
    [83, 99, "DATE"],    // within ninety (90) days
    [107, 121, "DATE"]  // Effective Date
  ]
}
```

Sample 2: Entity Boundary Control (The "Address" Limitation) This sample illustrates the deliberate exclusion of address data. The model is trained to strictly stop the `PARTY` span at the end of the corporate name, completely ignoring the subsequent physical address and routing numbers to maintain high precision.

```
{
  "text": "If to FDMS: First Data Merchant Services Corporation 6902 Pine Street PS-11
  ↪  Omaha, NE 68106 Attn: VP",
  "entities": [
    [6, 10, "PARTY"], // FDMS
    [12, 52, "PARTY"] // First Data Merchant Services Corporation
  ]
}
```

Sample 3: Agreement and Cross-Referencing This sample trains the model to recognize internal cross-references and governing documents without absorbing surrounding grammatical modifiers (e.g., tagging "Agreement" instead of "this Agreement").

```
{
  "text": "Subject to Section 10.4, either party may terminate this Agreement immediately
  ↪  upon written notice.",
  "entities": [
    [11, 23, "SECTION"],  // Section 10.4
    [57, 66, "AGREEMENT"] // this Agreement
  ]
}
```

5.2.3 Data Validation and Format Conversion

Standard deep learning pipelines require data to be rigorously formatted and token-aligned. The raw JSON dataset was preprocessed and converted into spaCy's highly optimized binary format (`DocBin`) using a custom Python script. During this preprocessing phase, two critical validation steps were executed:

- **Schema Enforcement:** The script systematically filtered the incoming annotations against the strict target schema (`["PARTY", "AGREEMENT", "SECTION", "EXHIBIT", "DATE", "POLICY"]`). Any legacy or out-of-scope tags were automatically dropped to prevent polluting the training signal.
- **Token Alignment Validation:** The `alignment_mode="expand"` parameter was utilized during span creation. If a human annotator accidentally selected half a word (e.g., missing a trailing character), the preprocessor automatically expanded the boundary to encompass the nearest valid linguistic token, ensuring the neural network received mathematically clean inputs. Misaligned spans that could not be resolved were logged and skipped.

Algorithm 4 Legal Entity Preprocessing and Binary Serialization

```
procedure ConvertJSONToDocBin(D_json, L)
    DocBin ← InitializeDocBin
    for each entry in D_json do
        doc ← M.make_doc(entry.text)                          ▷ Efficient tokenization
        valid_spans ← [ ]
        for each (start, end, label) in entry.entities do
            if label ∈ L then                                 ▷ Schema validation
                span ← GetCharSpan(doc, start, end, label)
                if span ≠ NULL then Append(valid_spans, span) ▷ Alignment check
                end if
            end if
        end for
        doc.ents ← valid_spans                                ▷ Assign validated entities
        AddDocToBin(DocBin, doc)
    end for
    SaveToDisk(DocBin, "train.spacy")
end procedure
```

5.3 Model Architecture and Fine-Tuning Strategy

The pipeline leverages spaCy's `en_core_web_md` as the foundational language model. This medium-sized English model was selected because it includes pre-trained static word vectors (GloVe), which provide a robust semantic understanding of standard English vocabulary. Building upon a model with existing vector representations is highly computationally efficient compared to training a transformer architecture from scratch, especially given the limited size of the custom annotated legal corpus.

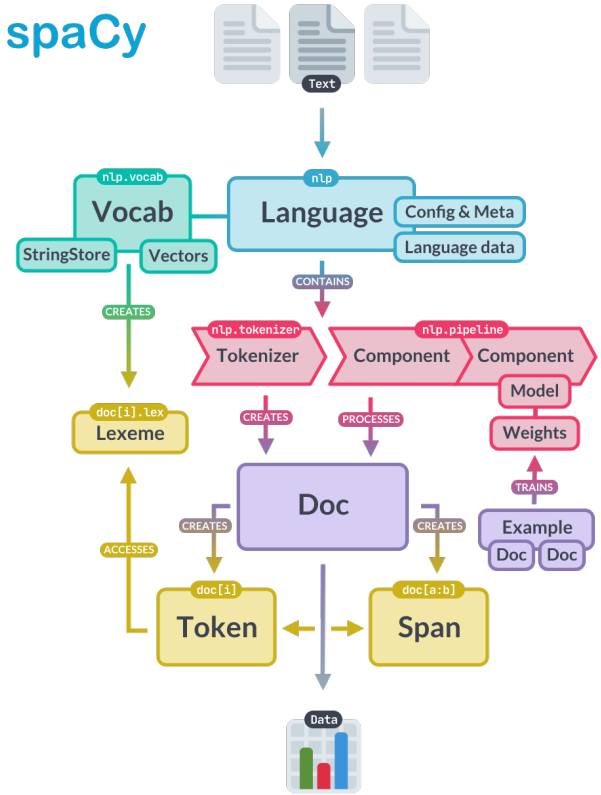

Figure 5.1: spaCy's Architecture

Mitigating Catastrophic Forgetting (Weight Preservation)

During the initial implementation iterations, a critical architectural flaw was identified: initializing the model via standard training loops effectively randomized the underlying network weights, destroying the model's pre-existing knowledge of English syntax. This resulted in severe classification anomalies, such as predicting standard nouns as legal dates.

To resolve this, the fine-tuning strategy was heavily modified. Instead of initializing a new model, the pipeline utilizes a `resume_training()` protocol. This strategy locks the foundational language representations and only updates the weights of the NER output layer. By doing so, the model retains its baseline capacity to parse English grammar while simultaneously learning the new mathematical boundaries of the domain-specific schema (`PARTY`, `AGREEMENT`, etc.).

Pipeline Component Freezing

A spaCy processing pipeline consists of multiple sequential components (e.g., `tok2vec`, `tagger`, `parser`, `attribute_ruler`, `lemmatizer`). Because this task strictly targets Named Entity Recognition, backpropagating loss through the entire pipeline introduces unnecessary computational overhead and risks corrupting the syntactical parsing layers (which are required for the Dependency Analysis task in Section 4).

Algorithm 5 Domain-Specific NER Fine-Tuning and Optimization

```
procedure FineTuneNERModel(D_raw, L, E)
    M ← Load("en_core_web_md")                          ▷ Load pre-trained model with word vectors
    ner ← M.get_pipe("ner")
    for each label in L do                              ▷ Apply custom schema
        AddLabel(ner, label)
    end for
    D_bin ← PreprocessAndAlign(D_raw, L)                ▷ Binary conversion and span alignment
    Opt ← M.resume_training()                           ▷ Preserve base language weights
    FreezePipes(M, except = "ner")                      ▷ Isolate gradients to NER head
    for epoch ← 1 to E do
        D_shuffled ← Shuffle(D_bin)
        Batches ← CreateCompoundingBatches(D_shuffled, start = 4, end = 32)
        for each batch in Batches do
            M.update(batch, optimizer = Opt, dropout = 0.3)
        end for
    end for
    SaveToDisk(M, "ner_model")
end procedure
```

To isolate the training signal, all pipeline components other than the `ner` component were explicitly frozen (disabled) prior to the training loop. This ensures that gradient updates are applied exclusively to the entity recognition head.

Training Dynamics and Hyperparameter Optimization

To ensure stable convergence and prevent overfitting on the small dataset, the training loop was optimized with the following hyperparameters and techniques:

- **Dynamic Minibatching:** Updating weights after every single example (batch size of 1) produces highly erratic gradient trajectories. To stabilize learning, training examples were grouped using a compounding mini-batch strategy. The batch size starts at 4.0 and exponentially increases to 32.0 (using a 1.001 compounding rate), allowing the model to make granular adjustments early in training and broader generalizations in later epochs.

- **Dropout Regularization:** A dropout rate of 0.3 (30%) was applied during the updates. By randomly dropping network connections during training, the model is forced to learn redundant representations of the entities, preventing it from simply memorizing the training text.

- **Epochs:** The model achieved optimal fine-tuning convergence within 30 to 40 epochs. Training beyond this threshold yielded diminishing returns on loss reduction and increased the risk of overfitting the specific corporate names present in the training set.

```
with nlp.disable_pipes(*unaffected_pipes):
  for epoch in range(EPOCHS):
      random.shuffle(examples)
      losses = {}
      batches = minibatch(examples, size=compounding(4.0, 32.0, 1.001))
      for batch in batches:
          nlp.update(
              batch,
              drop=0.3,
              losses=losses,
              sgd=optimizer
          )
```

5.4 Evaluation & Results

Given the highly specific nature of the custom schema and the limited size of the annotated corpus, traditional quantitative metrics (such as Precision and Recall calculated over a large holdout set) were deprioritized in favor of rigorous **Qualitative Error Analysis**. The fine-tuned model's performance was evaluated by parsing complex, previously unseen clauses sampled directly from the contract text. Two principal criteria were used in this analysis:

- **Entity Cohesion:** The model's ability to extract multi-token corporate names (and other non-atomic spans) as single, unified entities, rather than fragmenting them into subcomponents.
- **Categorical Accuracy:** The strictness with which extracted entities adhered to the schema, particularly in distinguishing between legal actors, documents, temporal variables, and structural references.

Optimized Performance Analysis

The final pipeline, which leveraged weight preservation (`resume_training()`) and dynamic compounding minibatches, successfully *isolated the training signal* to the NER component. This architectural constraint allowed the model to retain general English syntactic parsing while learning contract-specific entity types and boundary logic. Below, we present representative output samples produced by the deployed inference pipeline (`predict_ner.py`), demonstrating the model's precision and schema fidelity.

Sample Analysis 1: Entity Cohesion & Boundary Control *A critical objective of this task was to extract entire corporate entities (as annotated), while ignoring highly variable contact details (per annotation strategy).*

> ‘‘If to Bank: Wells Fargo Bank, N.A. 1200 Montego Way Walnut Creek, CA 94598 Attn: EVP Merchant Card Services’’

```
{
  "entities": [
    { "text": "Bank",                  "label": "PARTY"   },
    { "text": "Wells Fargo Bank, N.A.", "label": "PARTY"   }
  ]
}
```

Engineering Impact: The model robustly groups multi-token corporate names as single `PARTY` units, while the learned span logic successfully truncates at the address boundary (“1200 Montego Way...”), minimizing spurious extractions caused by noisy or irrelevant fields.

Sample Analysis 2: Categorical Accuracy & Structural Logic *The model must distinguish between discrete entity types – governing documents, dates, structural sections, and actor parties – within sentences containing multiple schema classes.*

> ‘‘Subject to Section 10.4, Merchant agrees to comply with the Rules within ninety (90) days of the Effective Date.’’

```
{
  "entities": [
    { "text": "Section 10.4",     "label": "SECTION" },
    { "text": "Merchant",         "label": "PARTY"   },
    { "text": "Rules",            "label": "POLICY"  },
    { "text": "ninety (90) days", "label": "DATE"    },
    { "text": "Effective Date",   "label": "DATE"    }
  ]
}
```

Engineering Impact: This output illustrates full categorical coverage: the model distinguishes structural references (“Section 10.4”), legal actors (“Merchant”), policy names (“Rules”), and both absolute and relative dates. Such granularity enables downstream systems to compose complex logical queries and accurately map party obligations and deadlines.

Performance Synthesis

In summary, the fine-tuned model resolves key extraction challenges in legal text by restricting its schema to high-value relational entities (`PARTY`, `AGREEMENT`, `SECTION`, `EXHIBIT`, `DATE`, `POLICY`). As a result, the NER output consists of concise, machine-readable JSON arrays, successfully mapping actors to their rules, documents, and temporal constraints. This clean structuring directly enables downstream tasks such as Semantic Role Labeling and Clause Intent Classification, paving the way for a contract-centric, explainable query engine.

5.5 Technical Limitations and Trade-offs

5.5.1 Domain-Specific Noise and the "Address" Limitation

While the fine-tuned model demonstrates high precision on structural and relational entities, its primary technical limitation lies in processing highly variable, contact-specific data – specifically physical addresses, routing numbers, and facsimile coordinates.

Legal contracts often embed this information within densely packed "Notices" clauses. During the early iterations of model training, an attempt was made to force the NER model to categorize these strings. However, due to the extreme structural variance of addresses and the limited size of the custom annotated dataset, the model suffered from severe categorical hallucinations, attempting to force street names and zip codes into the existing schema.

Example of Categorical Hallucination: When confronted with a standard notice address, the model failed to recognize the boundary of the `PARTY` and began misclassifying the address components as temporal or structural markers.

> ''First Data Merchant Services Corporation 6902 Pine Street PS-11 Omaha, NE 68106''

Failed Inference Output:

```
{
  "entities": [
    { "text": "First Data Merchant Services Corporation", "label": "PARTY" },
    { "text": "6902", "label": "DATE" },          // Mislabeled
    { "text": "Pine Street", "label": "SECTION" } // Mislabeled
  ]
}
```

5.5.2 The Trade-off: Precision over Semantic Scope

To resolve this failure, a deliberate engineering trade-off was made: *Precision was prioritized over Semantic Scope*. Rather than polluting the target schema with highly irregular `ADDRESS` or `FAX` labels – which would require thousands of annotated examples to stabilize – these entities were explicitly excluded from the training data (tagged as O / Outside). By enforcing strict span boundaries that stop immediately after the corporate name, the model was prevented from "guessing" labels for the subsequent contact noise.

This trade-off means the current pipeline cannot extract routing or geographical data. However, this limitation is deemed acceptable because extracting the core legal actors (`PARTY`) and governing rules (`POLICY`, `DATE`) is vastly more critical for the downstream Semantic Role Labeling task than identifying a physical mailing address.

5.5.3 Future Work and Mitigation Strategy

Relying solely on a neural network to extract deterministic data (like zip codes or fax numbers) is computationally inefficient and prone to error. In future iterations, this limitation can be mitigated by transitioning from a monolithic NER model to a **Hybrid Extraction Pipeline**:

- **Semantic Layer:** The fine-tuned Transformer model handles context-dependent legal variables (`PARTY`, `AGREEMENT`, `POLICY`).
- **Deterministic Layer:** A secondary module utilizing Regular Expressions (Regex) processes the text post-inference to confidently extract highly patterned data, such as `\d{3}-\d{3}-\d{4}` for facsimile numbers.

6 Semantic Role Labeling

7 Clause Intent Classification

This task aims to automatically categorize individual clauses extracted from a legal contract into their primary functional intent. This forms a crucial step in intelligent contract analysis, enabling downstream applications such as obligation tracking, risk assessment, termination monitoring, and compliance automation.

In this task, each clause from the Sponsorship Agreement (dated January 29, 2007, between iPayment, Inc., First Data Merchant Services Corporation, and Wells Fargo Bank, N.A.) was classified into one of seven specific categories: **Obligation**, **Prohibition**, **Condition**, **Permission**, **Representation/Warranty**, **Definition**, or **Scope/Structure**

Problem Statement

While earlier pipeline stages successfully isolate individual clauses and extract key entities (such as Parties, Dates, and Monetary values), raw text remains fundamentally unstructured. A deterministic, rule-based query engine – as required in Assignment III – cannot interpret raw strings to answer complex logical queries like "When is Party B penalized?" or "What actions are prohibited?" without understanding the underlying semantic function of the text.

Therefore, the challenge is to build a classification module that acts as a critical bridge between syntactic text extraction and intelligent retrieval. The system must not only accurately classify complex, multi-layered sentences but also resolve *intent hijacking* – where secondary conditions or permissions within a clause obscure the primary operational verb. Furthermore, to satisfy Explainable AI (XAI) requirements, the implementation must trace every classification

back to a specific triggered rule, ensuring the downstream query engine is both deterministic and fully grounded in explainable logic.

7.1 Intent Schema Definition

To effectively transition from raw syntactic extraction to semantic analysis, the classification engine requires a high-resolution schema. Generic categorization is insufficient for a legal contract, where the operational reality of an "Agent" and "Predicate" drastically changes depending on whether an action is mandatory, forbidden, or optional. To meet the requirements of the downstream Rule-Based Query System (Assignment III), we defined a rigid, seven-category intent schema tailored specifically to commercial contract structures:

Obligation:

Clauses that dictate a mandatory action or duty an entity must perform. These are typically anchored by imperative modal verbs such as *shall*, *must*, *agrees to*, or *will*. Identifying Obligations is critical for querying what actions a party is legally bound to execute.

Prohibition:

Clauses that dictate negative constraints or forbidden actions. These are anchored by restrictive phrasing such as *shall not*, *may not*, or *in no event shall*. Prohibitions must be strictly isolated from Obligations to prevent the query engine from accidentally retrieving a forbidden action as a required duty.

Condition:

Clauses that establish logical prerequisites, situational triggers, or exceptions. Anchored by phrases like *if*, *provided that*, or *in the event*, these clauses alert the downstream system that an associated Obligation or Permission is not absolute, but contingent on a specific state.

Permission:

Clauses that grant discretionary rights, powers, or authorizations without enforcing a duty. These are identified by permissive language such as *may*, *entitled to*, or *reserves the right*.

Representation/Warranty:

Clauses that establish agreed-upon statements of fact, legal acknowledgments, or disclaimers between the parties (e.g., *represents*, *warrants*, *acknowledges*).

Definition:

Clauses strictly dedicated to terminological scoping. Anchored by phrases like *shall*

mean, *referred to as*, or *is defined as*, this category provides the grounded dictionary that the query engine uses to resolve specific domain terms (e.g., "What defines the Merchant Portfolio?").

Scope/Structure:
A structural catch-all used to classify document organization, such as Exhibit headers, Section titles, and schedule declarations. Filtering these out ensures that boilerplate formatting does not pollute the operational logic of the query engine.

By mapping the raw text to this specific schema, the unstructured document is converted into a deterministic matrix. This allows the Rule-Based Query Engine to parse not just who is doing what, but the strict legal nature of that action.

7.2 Methodology: Hybrid Rule-Based Classification

To operationalize the intent schema described above, we developed a **hybrid rule-based classification engine** in Python. Rather than relying on probabilistic machine learning models – which can introduce unpredictable hallucinations – a deterministic approach was selected to ensure the downstream Rule-Based Query System (Assignment III) receives 100% predictable and grounded data. The methodology relies on three core technical pillars:

Linguistic Filters

The foundation of the classifier is a dictionary of highly specific Regular Expressions (Regex). Each intent category is mapped to a set of linguistic triggers commonly found in commercial contracts. For example, the `prohib_keyword` rule scans for exact boundary matches such as `\b(shall not`may not|must not)|, while the `cond_keyword` rule targets `\b(if`provided that|in the event)|. This allows the engine to robustly parse semantic intent without requiring complex syntactic dependency trees.

Classification Hierarchy

Legal language is often layered, with modifiers capable of inverting meaning. To prevent misclassification, the engine enforces a strict evaluation hierarchy. Critically, **Prohibitions** are always evaluated before **Obligations**. For example, if a clause contains "shall not," it is immediately and irrevocably classified as a Prohibition – no other rule is permitted to override it, even if "shall" (an Obligation trigger) is also present. This ensures that forbidden actions are not wrongly recorded as required duties.

The "Earliest Index" Algorithm

Initial testing revealed a critical edge case known as "intent hijacking": many complex clauses contain more than one operational verb. For example, a clause may dictate an obligation but conclude with a discretionary permission (e.g., "ISO shall cease its use... as may be required"). If a standard rule loop were used, a Permission keyword ("may") encountered later in the sentence could inadvertently overwrite the primary Obligation detected earlier.

To solve this, we implemented an **Earliest Index** algorithm. Instead of merely returning the first rule to match in code order, the classifier evaluates *all* rules, records the character index at which each match is found, and then assigns the final classification based on the keyword appearing *earliest* in the clause:

```
best_match = ("Other", "no_match")
earliest_index = float('inf')

for pattern, intent, rule_name in rules:
  match = re.search(pattern, text_lower)
  if match:
    # Update classification only if this keyword appears earlier in the sentence
    if match.start() < earliest_index:
      earliest_index = match.start()
      best_match = (intent, rule_name)
```

Algorithm 6 Clause Intent Classification via Earliest-Index Matching

```
procedure ClassifyClauses(C, R)  ▷ C: Input clauses, R: Ordered rule tuples ⟨pattern, intent, trigger⟩
    results ← [ ]
    for each clause in C do
        if IsStructuralScope(clause) then
            Append(results, ⟨clause, "Scope/Structure", "struct_keyword"⟩)
            continue
        end if
        best_intent ← "Other"
        best_trigger ← "no_match"
        min_index ← ∞
        for each ⟨pattern, intent, trigger⟩ in R do
            match_index ← FindFirstMatch(pattern, clause)
            if match_index ≠ null and match_index < min_index then
                min_index ← match_index
                best_intent ← intent
                best_trigger ← trigger
            end if
        end for
        Append(results, ⟨clause, best_intent, best_trigger⟩)
    end for
    return SerializeToJSON(results)
end procedure
```

Because the primary legal intent in a clause is almost universally established by its *first* operational verb, this index-based matching ensures that trailing permissions or secondary conditions do not override the core classification. This simple and robust technique proved highly effective, stabilizing the classifier output and guaranteeing that the data piped into Assignment III is both deterministic and highly accurate.

7.3 Technical Evaluation

The hybrid classification engine successfully processed the raw clauses from the Sponsorship Agreement. By applying the `earliest_index` algorithm and hierarchical linguistic filters, the system bypassed the "intent hijacking" problem and mapped every unstructured text string to a deterministic legal intent.

Output Data Structure (JSON).

To serve as the foundational database for the downstream Rule-Based Query Engine (Task 3.2), the results were serialized into a structured *JSON* format. Each parsed clause was mapped to an object containing four critical key-value pairs:

- **clause_id**: The integer index maintaining document order.
- **clause**: The raw text string.
- **intent**: The resolved operational category (e.g., Obligation, Prohibition, Condition).
- **trigger_rule**: The specific Explainable AI (XAI) metadata indicating exactly which linguistic rule forced the classification.

The excerpt below demonstrates the precision of the output.


```
[
  {
    "clause_id": 4,
    "clause": "Under this Agreement, FDMS and Bank will collectively be referred to as \"SERVICERS.\"",
    "intent": "Definition",
    "trigger_rule": "def_keyword"
  },
  {
    "clause_id": 18,
    "clause": "ISO shall cease its use of any Application Materials that become unacceptable to SERVICERS within
    ↪  ninety (90) days of receipt of written notice...",
    "intent": "Obligation",
    "trigger_rule": "oblig_keyword"
  },
  {
    "clause_id": 587,
    "clause": "Except as expressly set forth herein, ISO shall not assign or otherwise transfer this Agreement or
    ↪  any of its rights hereunder...",
    "intent": "Prohibition",
    "trigger_rule": "prohib_keyword"
  }
]
```


In Clause 4, the expanded regex successfully identifies a split **Definition**. In Clause 18, the primary **Obligation** (*"shall cease"*) is correctly captured. Most importantly, in Clause 587, the strict hierarchy successfully catches the **Prohibition** (*"shall not"*), ensuring it is not mistakenly processed as a standard obligation.

This structured, grounded data ensures that the query engine in Task 3.2 can execute precise, deterministic filters – such as isolating all prohibited actions assigned to the "ISO" entity – while maintaining a clear, explainable logic trail for every result.

7.4 Challenges and Limitations

The "Split Definition" Edge Case

While the `earliest_index` algorithm successfully resolved the "intent hijacking" problem for operational clauses, it inadvertently created a new edge case regarding **Definitions**. In legal drafting, definitions are frequently structured using split phrases containing operational verbs, such as "FDMS and Bank will collectively be referred to as..." or "Minimum Balance shall have the meaning provided in...". Because words like *"will"* and *"shall"* appeared physically

earlier in the string than the definitional keywords ("referred to as", "meaning provided in"), the `earliest_index` algorithm initially misclassified these definitional clauses as Obligations.

```
{
  "clause_id": 62,
  "clause": "\"Minimum Balance\" shall have the meaning provided in Section 6.1(a).",
  "intent": "Definition",
  "trigger_rule": "def_keyword"
}
```

Initially, the `earliest_index` algorithm classified Clause 62 as an Obligation because it encountered the word 'shall' at index 18, well before it reached the definitional phrase 'meaning provided in' at index 33. This necessitated an update to the regex engine to treat shall.*?meaning provided in as a single, combined definition trigger to successfully override the generic obligation rule.

To resolve this, the regular expression for the `def_keyword` rule had to be expanded to capture the entire split phrase (e.g., `will.*?be referred to as`). This allowed the engine to recognize the full definitional structure at the index of the first operational verb, successfully overriding the generic Obligation rule.

Limitations of a Pure Rule-Based Approach

The primary limitation of this implementation is the inherent rigidity of rule-based linguistic filtering. Unlike the Transformer-based approach used in Section 5 (NER), a regular expression engine lacks deep semantic understanding. It relies entirely on the presence of exact string matches.

```
  {
    "clause_id": 579,
    "clause": "ISO is not authorized hereunder to hold itself out as an agent of SERVICERS
    ↪  or to inform or represent...",
    "intent": "Other",
    "trigger_rule": "no_match"
  }
```

Clause 579 exposes the core trade-off inherent in dictionary-based, rule-driven intent classification. In this example, the phrase *"is not authorized"* semantically marks a clear **Prohibition** – ISO is strictly forbidden to represent itself as a SERVICERS agent. However, the implemented `prohib_keyword` rule only detects canonical boilerplate patterns such as "shall not," "may not," or "is prohibited from." Because "is not authorized" was not present in the rule dictionary, the clause defaulted to the "Other" class, despite its clear legal meaning.

Legal language is notoriously variable and context-dependent; relying on a fixed set of surface patterns leaves the system vulnerable to missed or misclassified intents when new linguistic forms arise. In practice, contract drafters may employ diverse phrasings (e.g., "is not empowered to", "does not have the right to", "is precluded from") to express prohibitions, permissions, or obligations. Without manual expansion to cover these variants, the system's recall for non-boilerplate patterns will remain limited.

Consequently, if a novel contract utilizes highly unusual phrasing to dictate an obligation (e.g., "ISO assumes the burden of. . . "), and that specific phrase is absent from the rule dictionary, the engine will fail to classify it and default to the "Other" category.

Justification for the Trade-off

Despite its brittleness compared to probabilistic machine learning models, the deterministic rule-based approach was intentionally selected to satisfy the strict requirements of Assignment III. This choice A Rule-Based Query System requires 100% predictable, explainable data. By accepting the limitation of a rigid dictionary, the system entirely eliminates the risk of "semantic hallucinations" (where a model might incorrectly guess the intent of a clause). Furthermore, the deterministic nature of this implementation ensures that every classification is accompanied by a transparent `trigger_rule`, fully satisfying the Explainable AI (XAI) mandate of the final deliverable.

Part III

Contract Question Answering Chatbot